

Web socket - Real time chat application

```
const wss = new WebSocketServer()
```

- `wss.on(connection, callback)` → so it is an entry point for any user connection

```
import express from "express";
import { WebSocketServer } from "ws";

// step 1: setup express server
const app = express();
const port = 8080;

app.listen(port, () => {
  console.log(`Server is running on the localhost port: ${port}`);
});

// step 2: setup a websocket server
const wss = new WebSocketServer({ server });

// step 3: listen for any incoming client requests
// wss.on(connection, ()=>{}) → so it is an entry point for any user in order to establish connection
// ws is an object that represents individual and unique connection for a single connection
wss.on("connection", (ws) => {
  console.log("New client has been connected!!");

  //listening from client from any messages
  ws.on("message", (message) => {
    console.log(`Received message: ${message}`);

    // For a chat app, we need to broadcast that message to every other connect
```

ed client.

```
//sending Broadcast message to all the connected clients
wss.clients.forEach((client) => {
  //we check if the client connection is still open before sending message
  if (client.readyState === ws.OPEN) {
    //converting from raw data(buffer) to string type
    client.send(message.toString());
  }
});

// handling client disconnecting
ws.on("close", () => {
  console.log("Client has disconnected");
});
});
```

Client side

- First of we need to initialise a WebSocket object

```
const port = 8080
const socket = new WebSocket('ws://localhost:8080')
// first thing we needs to do is to know if we are connected or not
socket.onopen = (event) => {
  console.log('Successfully connection to the websocket server');
}

socket.onmessage = (event) => {
  const messageList = document.getElementById('message-box')
  const newMessage = document.getElementById('messages')

  newMessage.textContent = event.data
}
```

```
messageList.appendChild(newMessage)
}
```

Now, what are the advantages of using the JSON protocol?

- Instead of just a piece of text, we can have a **structured package of information**.
- We can send multiple data at the same time.
 - For example. User, message and timestamp
- **Avoid confusion**. The client and server both know exactly what to expect. The username will always be in the `"username"` field.

Now, how to implement the JSON protocol?

1. Client needs to package the data into a JSON object before sending it
2. **The Server** will need to parse the incoming JSON string and then broadcast it.

Server-side modification for handling JSON format

1. Parsing incoming JSON into an object

```
// Inside the wss.on('connection', ...) callback:
ws.on('message', (messageAsString) => {
  // 1. Parse the incoming JSON string into an object.
  const messageObject = JSON.parse(messageAsString.toString());

  // Let's log it to see the structured data.
  console.log('Received message object:', messageObject);

  // 2. We can even add information to it before sending it back.
  // For example, a server timestamp.
```

```

messageObject.timestamp = new Date().toLocaleTimeString();

// 3. Stringify the object back to a JSON string before broadcasting.
const broadcastMessage = JSON.stringify(messageObject);

wss.clients.forEach(client => {
  if (client.readyState === ws.OPEN) {
    client.send(broadcastMessage);
  }
});
});

```

- Websocket is like a postal service, it can only send data like simple text or strings

1. Client Sends a Message

- **You have:** A JavaScript object in the browser.

```
{ username: "User42", text: "Hello!" }
```

- **The Problem:** You cannot send this object directly over the network.
- **The Solution:** You **pack it** into a string. `JSON.stringify()` turns the object into a text string:

```
'{"username":"User42","text":"Hello!"}'
```

- `socket.send()` mails this string to the server.

2. Server Receives the Message

- **The server gets:** The string `'{"username":"User42","text":"Hello!"}'`.
- **The Problem:** The server wants to work with this data (read the username, add a timestamp), but it's just a chunk of text.
- **The Solution:** It **un-packs** the string. `JSON.parse()` turns the string back into a usable JavaScript object:

```
{ username: "User42", text: "Hello!" }
```

- Now the server can easily add a timestamp:

```
{ username: "User42", text: "Hello!", timestamp: "10:29 AM" }
```

3. Server Broadcasts the Message to Everyone

- **The server has:** The updated JavaScript object.
- **The Problem:** The server now needs to send this object to all other connected clients. It faces the *exact same problem* the original client did—it can't send the object directly.
- **The Solution:** It must **re-pack** the object into a string for mailing. `JSON.stringify()` turns the updated object back into a string:

```
'{"username":"User42","text":"Hello!","timestamp":"10:29 AM"}'
```
- The server sends this new string to all clients.

In short, every time data crosses the network boundary, it must be in a string format.

- **To Send:** You `stringify` (Object → String).
- **To Use:** You `parse` (String → Object).

Feature Enhancements and Next Steps

1. Improve the User Experience (UX)

- **Persistent Usernames:** Right now, usernames are random. A great next step is to prompt a user for their name when they first join the chat. This would require you to send a new type of message, like `{ "type": "userJoined", "username": "Alice" }`, so the server can announce who has arrived.
- **Show a User List:** You could add a sidebar that lists everyone currently connected to the chat. To do this, the server would need to keep track of all connected usernames and send an updated list to all clients whenever someone joins or leaves.
- **"Is Typing..." Indicator:** When a user starts typing in the message box, they could send a `{ "type": "isTyping" }` message. When they stop, they could send a `{`

`"type": "stoppedTyping" }` message. This allows you to show that classic "Alice is typing..." notification to other users.

2. Make the Backend More Powerful

- **Implement Chat Rooms:** This is a big and rewarding challenge. Instead of one global chat, users could join different rooms (e.g., `ws://localhost:8080/chat/general` or `ws://localhost:8080/chat/gaming`). Your server would need to manage multiple groups of clients and only broadcast messages to the clients in the same room.
 - **Message Persistence:** Currently, all messages disappear if the server restarts. You could connect your server to a simple database (like **SQLite**) to save all messages. When a new user joins, you could send them the last 10-20 messages so they have some context of the conversation.
 - **Handle Reconnections:** What happens if a user's internet briefly drops? You could implement logic on the client to automatically try and reconnect to the server.
-

3. Deployment

- **Go Live!** You could deploy your application to the internet so your friends can use it. Services like **Glitch**, **Replit**, or **Heroku** provide free tiers that are perfect for hosting a small Node.js WebSocket application.

Improvement 1

- So, giving a username to chat feels more personal,
- we'll need a way for the server to distinguish between a regular chat message and a message that says "a new user has joined." This is where our JSON protocol becomes really powerful. We can add a `"type"` property to our message objects.

Implementing Message size limits

- So there is this thing called `maxPayload` which helps to set a limit for the payload that coming from the client (messages in our user case, because typically the size of the message is less than 1024 bytes or 1 KB)

```
const wss = new WebSocketServer({
  server,
  verifyClient: (info, done) => {
    // ... your existing origin validation logic ...
  },
  // Add this line to limit message size to 1KB
  maxPayload: 1024
});
```

Missing Production Features

Security

- ✓ ~~No input validation/sanitization~~
- ✓ ~~No rate limiting~~
- ✓ ~~No origin validation~~
- ✓ ~~No message size limits~~
- ✓ ~~No authentication~~

Error Handling

- ✓ ~~No reconnection logic on client~~
- ✓ ~~Limited error handling~~
- ✓ ~~No graceful shutdown~~

Configuration

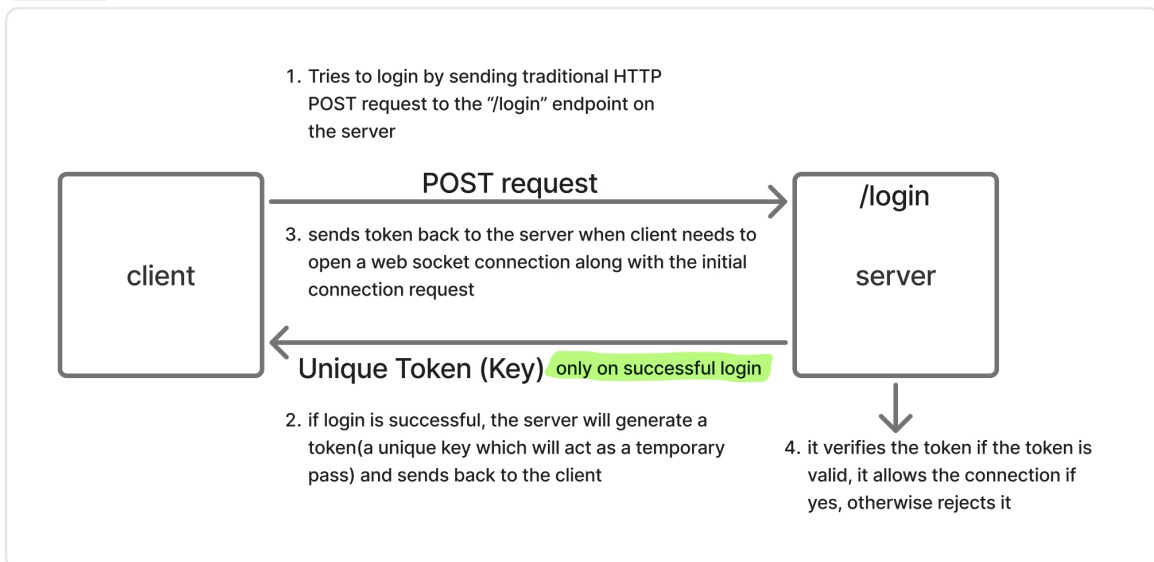
- ✓ ~~Hardcoded port (8080)~~
- ✓ ~~No environment variables~~
- ✓ ~~No logging framework~~

TODO

Authentication

- It is not a typical normal website, it is a websocket chat application so it doesn't need to authenticate on every single message, it's connection needs to be authenticated on the very first request from the client
- The easiest way or method for authentication is token based authentication

Section 1



1. first approach i found (but doesn't work properly)

```
app.post("/login", (req, res) => {  
  const { username, password } = req.body;  
  
  if (!username || !password) {
```



```

    return res
      .status(400)
      .json({ message: "username and password are required" });
  }

  const user = users[username];

  if (!user || user.password !== password) {
    return res
      .status(401)
      .json({ message: "Invalid username or password!" });
  }

  //let token = "";

  //crypto.generateKey("aes", { length: 512 }, (err, key) => {
  // if(err) throw err
  // token = key.export().toString('hex')
  //});

  // here the token is still empty, because the generateKey() function is an
  // async function that takes some time to be executed and move to the next li
ne
  //res.send({ token });

  //correct way
  crypto.generateKey("", { length: 512 }, (err, key) => {
    if(err) throw err
    const token = key.export().toString('hex')
    res.send({ token })
  })

  // or
  const token = crypto.randomBytes(32).toString('hex');

```

```
res.json({ token: token, username: username });  
});
```

```
// Define the WebSocket connection logic in its own function  
// It now takes the token and username as arguments  
function connect(token, username) {  
  socket = new WebSocket(`ws://localhost:8080?token=${token}`);  
  
  socket.onopen = () => {  
    console.log("WebSocket connection established and authenticated.");  
    reconnectionInterval = 1000;  
    setStatus(true);  
    currentUser.textContent = username;  
    // NOTE: We don't need to send a 'login' message anymore,  
    // because the server will identify us by our token.  
  };  
  
  // All your other socket.onmessage, socket.onclose, etc. handlers go here...  
  socket.onclose = () => {  
    // The reconnection logic should also be updated to re-authenticate  
    // For now, we can simplify it or tackle that next.  
  };  
}  
  
// --- Start the app by logging in first ---  
const loginDetails = {  
  username: "alice",  
  password: "123456"  
};  
  
fetch("/login", {  
  method: "POST",
```

```

    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(loginDetails),
  })
  .then(response => {
    if (!response.ok) {
      throw new Error('Login failed!');
    }
    return response.json();
  })
  .then(data => {
    // SUCCESS! We have the token. NOW we can connect.
    connect(data.token, data.username);
  })
  .catch(error => {
    console.error("Authentication failed:", error);
    statusText.textContent = "Authentication Failed";
    setStatus(false);
  });

```

```

const wss = new WebSocketServer({
  server,
  verifyClient: (info, done) => {
    const origin = info.origin;

    // check if the origin is in our allowedOrigins list
    if (!allowedOrigins.includes(origin)) {
      console.log(`Connection to origin: ${origin} rejected`);
      return done(false);
    }
    // else {
    //   // if the origin is in the allowedOrigins list, we still needs to check for t
    //   he token validation

```

```

// return done(true);
// }

if (info.req.url?.includes("token")) {
  const fullUrl = new URL(
    info.req.url,
    `http://${info.req.headers.host}`
  );
  const token = fullUrl.searchParams.get("token");

  if(!token || !activeToken.has(token)){
    console.log("Connection request error: invalid token");
    return done(false)
  }
  else{
    // find the username based on the entry from our map (activeToken)
    const username = activeToken.get(token);

    //attaching the username to the request object
    (info.req as any).username = username

    return done(true)
  }
}
},
//because the typical size of the chat message is less than 1024 kilobytes(1
kb)
  maxPayload: 1024,
});

```

Almost final server

```

import express from "express";
import { WebSocketServer, WebSocket } from "ws";
import rateLimit from "express-rate-limit";
import dotenv from "dotenv";
import crypto from "crypto";
import { URL, type Url } from "url";

dotenv.config();

const app = express();
const port = process.env.PORT || 3000;

const activeToken = new Map<String, String>();

interface ChatWebSocket extends WebSocket {
  username?: string;
}

function sanitize(str: string): string {
  return str
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}

const limiter = rateLimit({
  max: 200,
  windowMs: 60 * 60 * 1000,
  message: "Too many request from this IP address",
});

app.use(limiter);

```

```

const server = app.listen(port, () => {
  console.log(`Server is running on the localhost port: ${port}`);
});

const allowedOrigins = [
  "http://127.0.0.1:5500",
  "http://localhost:8080",
  "http://localhost:5500",
  "https://our-domain-app.com",
];

function getConnectedUsers(): string[] {
  const users: string[] = [];
  wss.clients.forEach((client: ChatWebSocket) => {
    if (client.readyState === WebSocket.OPEN && client.username) {
      users.push(client.username);
    }
  });
  return users;
}

//helper function to broadcast list of users to all the connected users(clients)
function broadcastUserList() {
  const connectedUsers = getConnectedUsers();
  const userListMessage = {
    type: "userList",
    users: connectedUsers,
  };

  wss.clients.forEach((client) => {
    if (client.readyState === WebSocket.OPEN) {
      client.send(JSON.stringify(userListMessage));
    }
  });
}

```

```

const wss = new WebSocketServer({
  server,
  verifyClient: (info, done) => {
    const origin = info.origin;

    // check if the origin is in our allowedOrigins list
    if (!allowedOrigins.includes(origin)) {
      console.log(`Connection to origin: ${origin} rejected`);
      return done(false);
    }
    // else {
    //   // if the origin is in the allowedOrigins list, we still needs to check for t
    he token validation
    //   return done(true);
    // }

    if (info.req.url?.includes("token")) {
      const fullUrl = new URL(
        info.req.url,
        `http://${info.req.headers.host}`
      );
      const token = fullUrl.searchParams.get("token");

      if (!token || !activeToken.has(token)) {
        console.log("Connection request error: invalid token");
        return done(false);
      } else {
        // find the username based on the entry from our map (activeToken)
        const username = activeToken.get(token);

        //attaching the username to the request object
        (info.req as any).username = username;

        return done(true);
      }
    }
  }
});

```

```

    },
    //because the typical size of the chat message is less than 1024 kilobytes(1
    kb)
    maxPayload: 1024,
  });

const users: Record<string, { password: string }> = {
  alice: { password: "123456" },
  bob: { password: "123456" },
};

app.post("/login", (req, res) => {
  const { username, password } = req.body;

  if (!username || !password) {
    return res
      .status(400)
      .json({ message: "username and password are required" });
  }

  const user = users[username];

  if (!user || user.password !== password) {
    return res
      .status(401)
      .json({ message: "Invalid username or password!" });
  }

  //simpler way
  const token = crypto.randomBytes(32).toString("hex");

  //because currently we are not using any database, so to make sure the ser
  ver remembers the valid token we are using Map DS
  // to store token and its username as a key value pair
  activeToken.set(token, username);

```



```

res.json({ token: token, username: username });

//another way
// crypto.generateKey("aes", { length: 512 }, (err, key) => {
//   if(err){
//     return res.status(500).json({ message: "Error generating token" })
//   }
//   const token = key.export().toString('hex')

//   res.json({ token: token, username: username })
// });

});

wss.on("connection", (ws: ChatWebSocket, req) => {
  console.log("New client has been connected!!");

  // console.log(req)

  ws.username = (req as any).username;

  const accouncement = {
    type: "accouncement",
    message: `${ws.username} || "unknown" has joined the chat room`,
  };

  wss.clients.forEach((client) => {
    if (client.readyState === ws.OPEN) {
      client.send(JSON.stringify(accouncement));
    }
  });

  // sending broadcast message to all the connected users/clients
  broadcastUserList();

  // listening for messages from specific client
  ws.on("message", (message) => {

```

```

console.log(`Received message: ${message}`);

try {
  const messageObject = JSON.parse(message.toString());
  if (messageObject.type === "chat") {
    if (
      !messageObject.message ||
      messageObject.message.length > 250
    ) {
      return;
    }
    const chatMessage = {
      username: ws.username,
      message: sanitize(messageObject.message),
      timestamp: new Date().toLocaleTimeString(),
    };

    wss.clients.forEach((client) => {
      if (client.readyState === ws.OPEN) {
        client.send(JSON.stringify(chatMessage));
      }
    });
  }
} catch (error) {
  console.log("Error Parsing JSON: ", error);
}

});

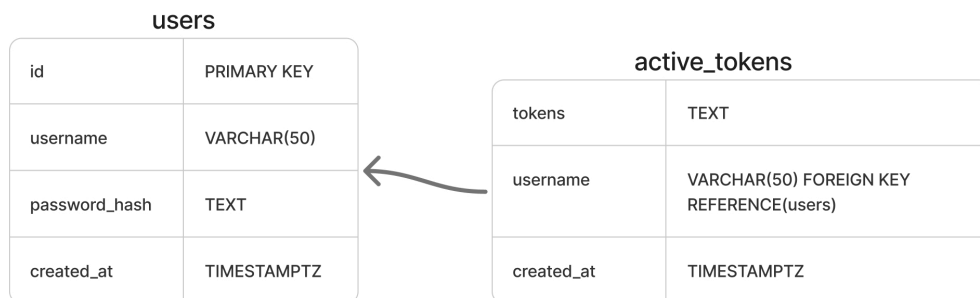
// handling client disconnecting
ws.on("close", () => {
  console.log("Client has disconnected");

  setTimeout(() => {
    broadcastUserList();
  }, 100);
});

```

```
});  
});
```

Database schema



Question: Why to use connection-pool in order to connect to the postgres DB?

- Connecting a new client to the PostgreSQL server requires a handshake which can take 20-30 milliseconds. During this time passwords are negotiated, SSL may be established, and configuration information is shared with the client & server. Incurring this cost *every time* we want to execute a query would substantially slow down our application.

- **Client** class basically used to establish a single and dedicated connection to the database
- **Client** class works like assigning new bike to every single individual who wants to ride it. but the main drawback is that it is slow and inefficient
- **Pool** is like having a fleet of taxis waiting at a stand. When a request comes in, you grab an available taxi. When it's done, the taxi returns to the stand, ready for the next request. It's much faster.
- **Postgres server can handle only the limited number of requests/connections**

improvements

- ☐ Implementing jwt for security purposes

Implement JWT with expiration

- **Problem:** if anyone gets access to the token, they have permanent access to that user account
- **Solution:** JSON web token(JWT) - it is a special token that contains data and most importantly, an expiration date

Steps:

1. Install the `jsonwebtoken` library: `npm install jsonwebtoken @types/jsonwebtoken`
2. in our '/login' route, instead of using crypto function to generate token, we can use `jwt.sign()` function

```
import jwt from 'jsonwebtoken';
const secret = process.env.JWT_SECRET;
```

```
const token = jwt.sign({ username }, secret, { expiresIn: '1h' });
```

3. In your `verifyClient` function, you would *verify* the token. The `jwt.verify` function automatically checks the signature and the expiration date.

```
jwt.verify(token, secret, (error, decoded) => {
  if (error) {
    console.log("Token verification error", error.message)
    return done(false)
  }
  pool
    .query("SELECT * from active_tokens WHERE token= $1", [token])
    .then((result) => {
      if (result.rows.length == 0) {
        //meaning the token not found in DB, reject the connection
        request
          done(false);
      } else {
        (info.req as any).username = result.rows[0].username;
        done(true);
      }
    })
    .catch((error) => {
      console.log("Token verification error: ", error);
      done(false);
    });
});
```

Graceful shutdown

- What is we accidentally shutdown the server in middle of the session where users are still in the chat session?

The Problems with an Abrupt Stop 🛑

Think of it like a store manager just walking out and locking the doors at 3 PM without telling any staff or customers. It creates chaos.

- **Database Connection Leaks:** When your app stops abruptly, the database server doesn't know you're gone. It keeps the connections you were using open, thinking they're still active. If you restart your server many times, these "ghost" connections build up until the database hits its connection limit and refuses to let anyone else in, crashing your application. This is a classic **resource leak**.
- **Bad User Experience:** For the users, the chat just freezes. They get a generic, ugly "connection error" message from their browser. They don't know if the server crashed, if their internet went out, or if they were kicked. It feels broken and unprofessional.

So this is how graceful shutdown works

- it ensures that everything is cleaned up before closing the connection or server

A graceful shutdown is an orderly, polite procedure for closing the server. It ensures everything is cleaned up properly before the application process actually exits.

It follows a sequence like this:

1. **Stop Accepting New Connections:** The server stops listening for new HTTP requests and Web Socket connections. It's like locking the front door of the store but letting the people inside finish their business.
2. **Notify Existing Clients:** It sends a final message to all connected chat users, like `"Server is restarting for maintenance..."`. This allows the front-end to display a helpful message instead of a generic error.

3. **Close All Connections:** It systematically closes everything that's open:
 - It tells the **HTTP server** to finish any pending requests and then close.
 - It tells the **Web Socket server** to close all active client connections.
 - It tells the **database pool** to close all its connections.
4. **Exit the Process:** Only after all the cleanup is done, the Node.js process exits cleanly.

This prevents database leaks and gives users a much smoother experience.

we can achieve it by listening to process signals

- so the most important signals for shutdown are
1. SIGINT (signal Interrupt): this what your terminal sends to the app when you do (ctrl+c)
 2. SIGTERM (Signal termination): This is the standard, "polite" signal that process managers (like Docker, Kubernetes, or PM2) send to ask your application to shut down cleanly.

```
function GraceFullShutdown() {
  console.log("Shutdown signal received, starting graceful shutdown.....");

  // 1. stopping http server from receiving new connections
  server.close((err) => {
    if (err) {
      console.log("Error during HTTP server shutdown: ", err);
      process.exit(1);
    }

    console.log("HTTP server closed");

    // 2. closing the websocket server
    wss.close();
    console.log("Websocket server ended");

    // 3. closing the database connection
    pool.end(() => {
      console.log("Database pool closed");
      // 4. exit the process clearly
      console.log("Graceful shutdown complete.");
      process.exit(0);
    });
  });
}

process.on("SIGINT", GraceFullShutdown);
process.on("SIGTERM", GraceFullShutdown);
```

proper logging

- ✓ **Track Each Client:** When a client connects, we'll create a simple counter for them to track their message activity.
- ✓ **Increment on Message:** Every time the client sends a message, we'll increment their counter.
- ✓ **Check the Limit:** If the message count exceeds our defined limit within a certain time frame (e.g., more than 20 messages in 10 seconds), we'll consider it spam.
- ✓ **Enforce:** We'll disconnect the spamming client by closing their Web socket connection.
- ✓ **Reset Periodically:** We'll reset the counter for every client periodically (e.g., every 10 seconds) to allow them to send more messages.


```

wss.on("connection", (ws: ChatWebSocket, req) => {
  logger.info(`New client has been connected: ${ws.username}`);
  ws.username = (req as any).username;

  // --- Start of Corrected Rate Limiting Logic ---

  let messageCounter = 0;
  // 1. Give the timer a variable name so we can clear it later
  const rateLimitTimer = setInterval(() => {
    // 2. Actually reset the counter inside the callback
    messageCounter = 0;
  }, rateLimitInterval); // 3. Fix the variable name typo

  // --- End of Corrected Rate Limiting Logic ---

  const announcement = {
    type: "announcement",
    message: `${ws.username || "unknown"} has joined the chat room`,
  };

  wss.clients.forEach((client) => {
    if (client.readyState === ws.OPEN) {
      client.send(JSON.stringify(announcement));
    }
  });

  broadcastUserlist();

  ws.on("message", (message) => {
    // This part was mostly correct
    messageCounter++; // Increment before checking
    if (messageCounter > rateLimit) {
      logger.warn(`Rate limit exceeded for ${ws.username}. Disconnecting.`);
    };

    ws.close(1008, "You are sending messages too frequently.");
  });

```

```

        return;
    }

    logger.debug(`Received message from ${ws.username}: ${message}`);
    try {
        // ... (rest of your message handling logic)
    } catch (error) {
        logger.error("Error Parsing JSON: ", error);
    }
});

ws.on("close", () => {
    logger.info(`Client has disconnected: ${ws.username}`);

    // --- Start of Rate Limiting Cleanup ---
    // 4. IMPORTANT: Clean up the timer to prevent memory leaks
    clearInterval(rateLimitTimer);
    // --- End of Rate Limiting Cleanup ---

    setTimeout(() => {
        broadcastUserlist();
    }, 100);
});
});

```

Implementing HTTPS & WSS(web-socket secure)

- Our server is currently running on `http://` and `ws://`
- In production, all traffic must be encrypted using `https://` and `wss://`

- Sending login credentials over an unencrypted connection is a major security vulnerability
- So we need to set up a reverse proxy like Nginx or caddy
- Reverse proxy will sit in front of your Node.js application, handle the SSL/TLS certificates and forward the encrypted traffic to your app running on localhost

